

Reliable Data Delivery

Objectives

- Replication
- Broker configuration
- Ack
- Validating reliability

Reliability Guarantees from Kafka

- Kafka provides order guarantee of messages in a partition.
- Produced messages are considered “committed” when they were written to the partition on all its in-sync replicas (but not necessarily flushed to disk).
- Producers can choose to receive acknowledgments of sent messages when the message was fully committed, when it was written to the leader, or when it was sent over the network.
- Messages that are committed will not be lost as long as at least one replica remains alive.
- Consumers can only read messages that are committed.

Replication Recap

- Message written in multiple replicas is how Kafka provides durability of messages in the event of a crash.
- Each Kafka topic is broken down into partitions
- Each partition can have multiple replicas, one of which is a designated leader.
- All events are produced to and consumed from the leader replica. Other replicas just need to stay in sync with the leader and replicate all the recent events on time.
- If the leader becomes unavailable, one of the in-sync replicas becomes the new leader.
- A replica is considered in-sync
 - if it is the leader for a partition
 - if it is a follower then, otherwise it is considered out-of-sync
 - Has an active session with Zookeeper
 - Fetched messages from the leader in the last 10 seconds
 - Fetched the most recent messages from the leader in the last 10 seconds
- With fewer in-sync replicas, the effective replication factor of the partition is lower and therefore there is a higher risk for downtime or data loss

Broker Configuration

- Replication Factor
 - The topic-level configuration is `replication.factor`. At the broker level, you control the `default.replication.factor` for automatically created topics.
- Unclean Leader Election
 - This configuration is only available at the broker (and in practice, cluster-wide) level.
 - The parameter name is `unclean.leader.election.enable` and by default it is set to `true`
 - As discussed earlier, when the leader for a partition is no longer available, one of the in-sync replicas will be chosen as the new leader. This leader election is “clean” (no loss of committed data, and committed data exists on all in-sync replicas)
 - Setting `unclean.leader.election.enable` to `true` means we allow out-of-sync replicas to become leaders (knowns as unclean election)
- Minimum In-Sync Replicas
 - Both the topic and the broker-level configuration are done using `min.insync.replicas`
 - Why – When we are left with single replica choice between availability and consistency is difficult, hence always keep more than one as in-sync.
 - If `min.insync.replicas` is not in place then producers that attempt to send data will receive `NotEnoughReplicasException`

Using Producers in a Reliable System - Ack

- Send Acknowledgments – Producers can choose between three different acknowledgment modes
 - `acks=0` means that a message is considered to be written successfully to Kafka if the producer managed to send it over the network.
 - `acks=1` means that the leader will send either an acknowledgment or an error the moment it got the message and wrote it to the partition data file (but not necessarily synced to disk).
 - `acks=all` means that the leader will wait until all in-sync replicas got the message before sending back an acknowledgment or an error. In conjunction with the `min.insync.replica` configuration on the broker
- Configuring Producer Retries
 - Two parts to handling errors in the producer: the errors that the producers handle automatically
 - Retriable – Example `LEADER_NOT_AVAILABLE` error
 - Nonretriable – Example `INVALID_CONFIG` error
- Additional Error Handling – Developer must handle
 - Nonretriable broker errors such as errors regarding message size, authorization errors, etc.
 - Errors that occur before the message was sent to the broker—for example, serialization errors
 - Errors that occur when the producer exhausted all retry attempts, available memory

Using Consumers in a Reliable System

- Data is only available to consumers after it has been committed to Kafka – meaning it was written to all in-sync replicas. The only thing consumers are left to do is make sure they keep track of which messages they've read and which messages they haven't.
- Important Consumer Configuration Properties for Reliable Processing
 - The first is `group.id`
 - The second relevant configuration is `auto.offset.reset` (earlier/latest)
 - The third relevant configuration is `enable.auto.commit` (true/false)
 - The fourth relevant configuration is tied to the third, and is `auto.commit.interval.ms`
- Explicitly Committing Offsets in Consumers
 - Always commit offsets after events were processed
 - Commit frequency is a trade-off between performance and number of duplicates in the event of a crash
 - Make sure you know exactly what offsets you are committing
 - Design your application keeping Rebalances in view

Validating System Reliability

- Validating Configuration
- Validating Applications
- Monitoring Reliability in Production

Summary

- Replication
- Broker configuration
- Ack
- Validating reliability